

"Trust the token, not the network — every hop is a new perimeter."

SUPPLEMENTARY · MODULE 7

Microservices Security

Secure service-to-service and user paths: OAuth2/OIDC, JWT, mTLS, secrets on OpenShift, and the controls that satisfy banking audit.



SECURITY THREATS IN DISTRIBUTED SYSTEMS

Unauthorized access, insecure service communication, injection, and misconfiguration top the list of threats a distributed architecture introduces.



COMMON THREATS

- Unauthorized access, insecure service-to-service communication, vulnerable components
- Data exposure, injection attacks, denial of service
- Supply chain attacks and misconfiguration

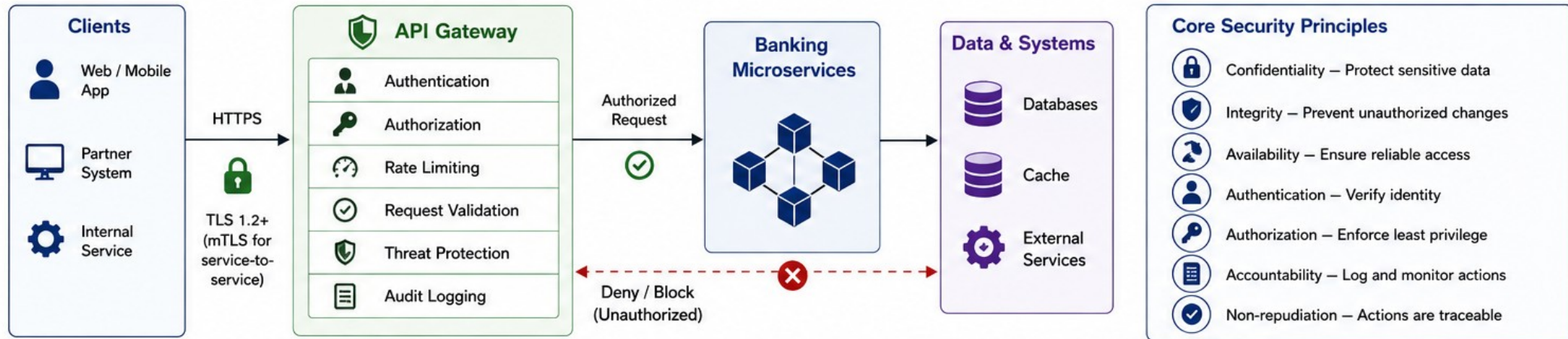


BEST PRACTICES TO REDUCE RISK

- Enforce strong authentication and authorization; encrypt data in transit and at rest
- Implement network policies, manage secrets securely, monitor and alert continuously

KEY TAKEAWAY Unauthorized access, insecure service communication, injection, and misconfiguration top the list of threats a distributed architecture introduces.

Security Fundamentals for Banking APIs



- ### 1. Authentication
- Use OAuth 2.1 / OpenID Connect
 - Access Tokens (JWT)
 - Short-lived tokens
 - Multi-Factor Authentication (MFA) for users
 - mTLS for service-to-service

- ### 2. Authorization
- Enforce Role-Based Access Control (RBAC)
 - Scopes and Permissions
 - Least Privilege Access
 - Authorize every request
 - Do not trust client-side checks

- ### 3. Transport Security
- Enforce HTTPS everywhere
 - TLS 1.2 or higher
 - HSTS for browsers
 - mTLS between services
 - Secure ciphers and certificates

- ### 4. Data Protection
- Encrypt sensitive data at rest (AES-256)
 - Encrypt data in transit (TLS)
 - Tokenize / mask PAN and PII
 - Never store secrets in code
 - Use a secrets manager

- ### 5. Input Validation
- Validate all inputs (server-side)
 - Use allowlists
 - Prevent injection (SQL, NoSQL, Command)
 - Validate schema and data types
 - Normalize and sanitize data

- ### 6. Protection & Resilience
- Rate limiting and throttling
 - DDoS protection
 - Circuit breaker and timeouts
 - WAF rules for common attacks
 - Prevent abuse and brute force

- ### 7. Logging, Monitoring & Audit
- Log security events and errors
 - Include correlation IDs
 - Monitor in real time
 - Alert on suspicious activity
 - Retain logs as per policy

- ### 8. API Security Best Practices
- Use API Gateway
 - Version your APIs
 - Follow secure coding practices
 - Regular dependency updates
 - Perform security testing

API Security Checklist

- ✓ Authenticate every request
- ✓ Authorize with least privilege
- ✓ Encrypt data in transit & at rest
- ✓ Validate input and output
- ✓ Limit rate and prevent abuse
- ✓ Log, monitor and audit
- ✓ Protect secrets and credentials



Remember

In banking, security is not optional.
Protect data. Protect customers. Build trust.

DEFENSE IN DEPTH FOR MICROSERVICES



Multiple, overlapping security controls across people, process, and technology

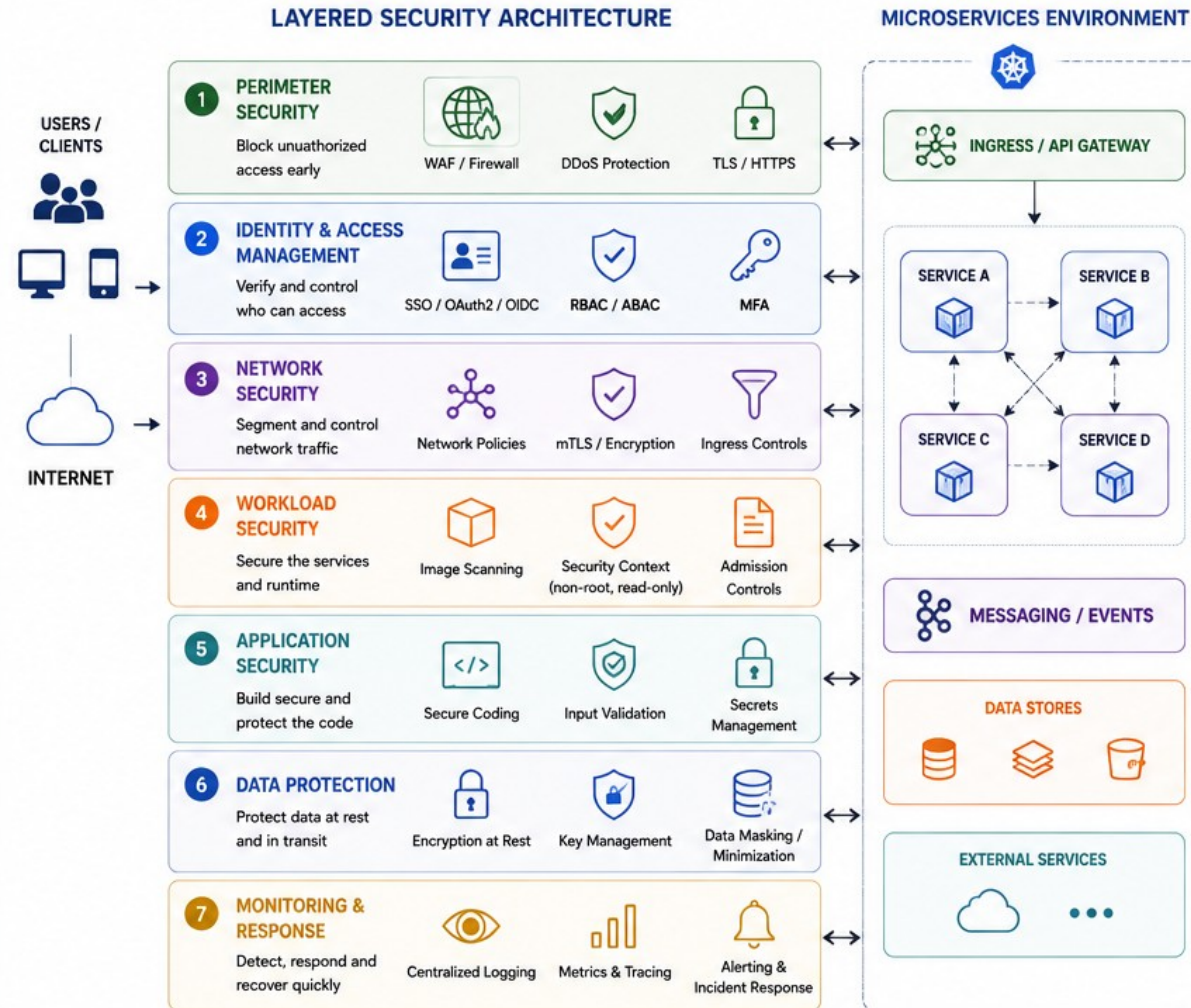
PRINCIPLE

Assume breaches will happen.
Layer multiple controls so that if one fails, others continue to protect your systems and data.

- ✓ No single point of failure
- ✓ Reduce blast radius
- ✓ Protect data and services
- ✓ Detect and respond quickly
- ✓ Continuous improvement

KEY TAKEAWAYS

- Apply controls at every layer of the microservices stack.
- Combine preventive, detective, and responsive controls.
- Automate and enforce security policies consistently.
- Monitor, audit, and improve continuously.



DEFENSE LAYERS EXPLAINED

- 1 PERIMETER SECURITY**
WAF, firewalls, DDoS protection, TLS termination, IP allowlists.
- 2 IDENTITY & ACCESS MANAGEMENT**
Strong authentication, SSO, least privilege access, MFA.
- 3 NETWORK SECURITY**
Network segmentation, zero trust between services, restrict traffic with network policies.
- 4 WORKLOAD SECURITY**
Harden containers and pods, scan images, enforce security standards at runtime.
- 5 APPLICATION SECURITY**
Secure coding, validate inputs, manage secrets, protect APIs and endpoints.
- 6 DATA PROTECTION**
Encrypt data, manage keys, limit exposure, classify and minimize data.
- 7 MONITORING & RESPONSE**
Centralized logs, metrics, traces, alerts, and automated response playbooks.



BEST PRACTICES



Apply least privilege access everywhere.



Automate security checks in CI/CD.



Continuously monitor and detect anomalies.



Patch, update, and remediate continuously.



Test and validate your defenses.



Build a security-first culture and mindset.

AUTHENTICATION VS. AUTHORIZATION · IDENTITY AND ACCESS MANAGEMENT · OAUTH 2.0 FUNDAMENTALS



AUTHENTICATION: WHO ARE YOU?

- **Verifies identity:** username/password, MFA, OAuth2/OIDC login, API key or certificate



AUTHORIZATION: WHAT CAN YOU DO?

- **Checks permissions:** roles, scopes in an access token, ACLs, resource-level permissions
- First prove who you are, then determine what you can do



HOW IAM WORKS

- Identities (users, services, apps, devices) -> Authentication -> Authorization -> Access
- Continuous monitoring wraps around the entire flow



COMMON IAM CAPABILITIES

- Single sign-on (SSO), multi-factor authentication, RBAC, ABAC
- Privileged access management (PAM), identity federation and social login



CORE ROLES

- **Resource owner:** the user who owns the data
- **Client:** the application requesting access on the user's behalf
- **Authorization server:** authenticates the user and issues access tokens
- **Resource server:** hosts the protected resources



BEST PRACTICES

- Use least-privilege scopes; always use HTTPS; validate tokens on the resource server
- Keep tokens short-lived and never store sensitive data in an access token

KEY TAKEAWAY A customer may prove who they are through login, but authorization is the separate check that decides which accounts they can actually view. • Identities, authentication, authorization, and access flow through one governed framework -- with audit and monitoring wrapped around all of it. • Resource owner, client, authorization server, and resource server are OAuth2's four roles -- each with one clear responsibility.

OAuth 2.0 and OpenID Connect

What They Are



OAuth 2.0

Authorization framework that enables apps to obtain limited access to user resources.



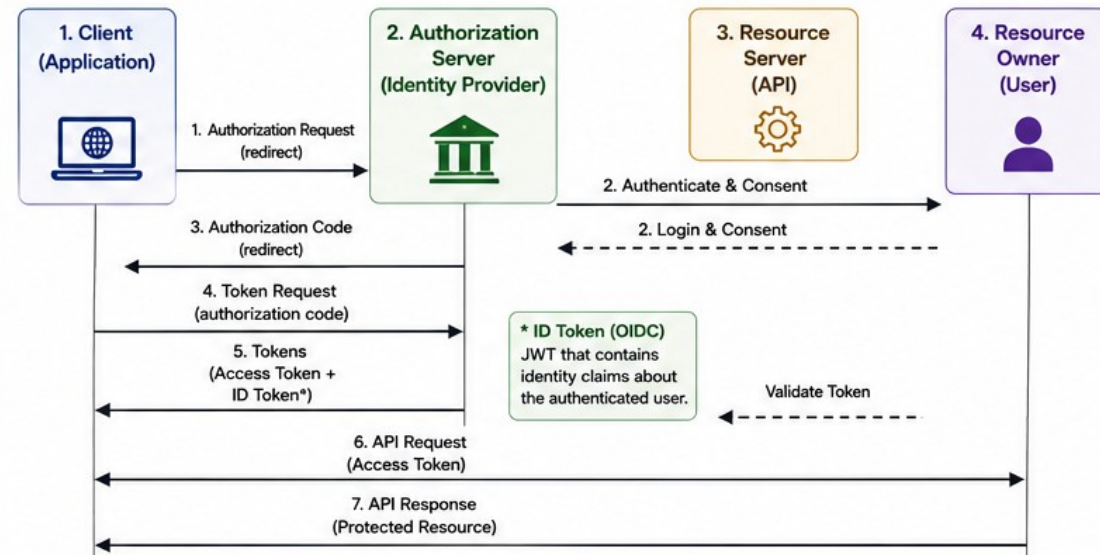
OpenID Connect (OIDC)

Identity layer on top of OAuth 2.0 that adds authentication and standardized user info.

In Short

OAuth 2.0 = Authorization
OIDC = Authentication + User Identity

OAuth 2.0 / OIDC Authorization Code Flow (with OIDC)



Key Concepts



Client
Application requesting access to resources on behalf of the user.



Authorization Server (AS)
Authenticates the user, obtains consent, and issues tokens.



Resource Server (RS)
Hosts the protected APIs and validates access tokens.



Resource Owner (User)
The end-user who owns the data.



Access Token
Opaque or JWT token used to access APIs.



ID Token (OIDC)
JWT that contains user identity claims (e.g., name, email, sub).

OAuth 2.0 Grant Types (Common)



Authorization Code (with PKCE)

Used by web & mobile apps. Most secure for user-facing apps.



Client Credentials

Used by server-to-server applications. No user involved.



Refresh Token

Used to obtain new access tokens when the current one expires.



Implicit (Deprecated)

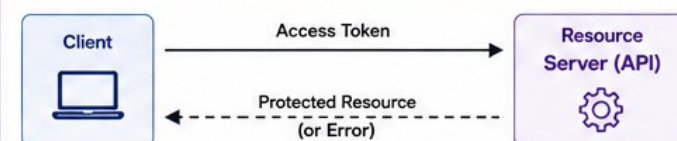
Not recommended. Use Authorization Code with PKCE instead.

ID Token (OIDC) – Example Claims (JWT Payload)

```
{
  "iss": "https://idp.example.com",
  "sub": "248289761001",
  "aud": "banking-app-client-id",
  "exp": 1715689200,
  "iat": 1715685600,
  "nonce": "n-0S6_WzA2Mj",
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "preferred_username": "janedoe",
  "auth_time": 1715685500
}
```

Claim	Description
iss	Issuer (Authorization Server)
sub	Subject (User Identifier)
aud	Intended Audience (Client ID)
exp	Expiration Time
iat	Issued At
nonce	Used to prevent replay attacks
name	Full name
email	Email address
preferred_username	Username
auth_time	Authentication time

Token Usage



Token Validation at Resource Server

- ✓ Validate signature (if JWT)
- ✓ Validate issuer, audience, expiration
- ✓ Validate scopes/permissions
- ✓ Process request if valid

Benefits



Secure delegated access



User authentication with single sign-on



Fine-grained access control



Token-based & revocable access



Industry standard & widely adopted

Best Practices

- ✓ Use Authorization Code + PKCE for public clients
- ✓ Use short-lived access tokens
- ✓ Store tokens securely
- ✓ Use HTTPS everywhere
- ✓ Validate tokens at the resource server
- ✓ Request only necessary scopes
- ✓ Use refresh tokens securely
- ✓ Revoke tokens when no longer needed

OPENID CONNECT FUNDAMENTALS · OAUTH ROLES AND AUTHORIZATION FLOWS



TOKENS IN OIDC

- **ID Token (JWT):** identity information about the authenticated user (sub, name, email)
- **Access Token:** used to access protected resources (APIs)
- **Refresh Token:** used to obtain new access tokens without user interaction



KEY BENEFITS

- Single sign-on (SSO) across applications, standardized identity information
- Delegated access with OAuth2, reduces custom authentication implementations



FLOW SELECTION GUIDE

- **Web application (server-side) / Mobile / Native:** Authorization Code (with PKCE)
- **Single-page application (SPA):** Authorization Code with PKCE
- **First-party/trusted app:** Resource Owner Password Credentials
- **Microservice / backend-to-backend:** Client Credentials

KEY TAKEAWAY The ID token is what OIDC adds: a JWT carrying identity claims like sub, name, and email that OAuth2 alone never provided. • Web and mobile apps use authorization code with PKCE; server-to-server calls use client credentials -- picking the wrong flow weakens security.

AUTHORIZATION CODE FLOW WITH PKCE · CLIENT CREDENTIALS FLOW · ACCESS TOKENS, ID TOKENS, AND REFRESH TOKENS

HOW PKCE WORKS

- 1. Client generates a high-entropy code_verifier
- 2. Client derives code_challenge = BASE64URL(SHA256(code_verifier))
- 3. Authorization request includes the code_challenge
- 4. Token request includes the original code_verifier for validation

HOW IT WORKS

- Client authenticates with the authorization server using its own credentials
- No user involvement -- the client accesses resources as itself
- **Common use:** microservice-to-microservice calls, scheduled jobs, background services

WHY PKCE MATTERS

- Essential for public clients (SPAs, mobile, native apps) with no client secret
- Now recommended for all OAuth2 clients per RFC 7636

TOKEN COMPARISON

FEATURE	ACCESS TOKEN	ID TOKEN	REFRESH TOKEN
Primary use	Access APIs	Authenticate user	Get new tokens
Sent to API	Yes	No	No
Lifetime	Short	Short	Long

KEY TAKEAWAY A stolen authorization code is useless without the original code_verifier -- that's the entire security value PKCE adds for public clients. • Transaction Service authenticates as itself, gets a token, and calls Account Service -- no end-user session involved anywhere. • Access tokens are short-lived and sent to APIs; ID tokens are never sent to APIs; refresh tokens are long-lived and stored securely by the client.

User-Delegated Access vs Workload Identity

User-Delegated Access
(On-Behalf-Of a User)

An application acts on behalf of a signed-in user, with the user's consent and the user's permissions.

1. User

Signs in (MFA)

2. Client Application

Requests access to APIs

3. Authorization Server (IdP)

Validates token and returns data

4. Resource Server (API)

Validates token and returns data

Authorization Request

Access Token (for user)

Token contains user context and permissions

```
{
  "iss": "https://idp.example.com",
  "sub": "user123",
  "aud": "api://banking-api",
  "scope": "accounts.read transactions.initiate",
  "roles": ["customer"],
  "name": "Jane Doe",
  "preferred_username": "jane.doe@example.com"
}
```

Common Use Cases

- Customer viewing account balances
- User initiating a payment
- User managing their profile
- Any action requiring the user's identity and consent

Key Characteristics

- Requires user authentication and consent
- Token represents the user (user context in token)
- Access is limited to the user's permissions
- Subject to user lifecycle and revocation
- Example: Mobile app, Web app calling APIs on behalf of a user

Workload Identity
(Application-to-Application)

An application (workload/service) accesses APIs using its own identity, without a user context.

1. Workload / Service

Daemon, service, or background job

2. Authorization Server (IdP)

Validates token and returns data

3. Resource Server (API)

Validates token and returns data

Token Request
(Client Credentials or Federation)

Access Token (for workload)

Token contains workload (client) identity, not a user

```
{
  "iss": "https://idp.example.com",
  "sub": "spn:banking-backend",
  "aud": "api://banking-api",
  "scope": "transactions.process",
  "roles": ["processor"],
  "appid": "banking-backend",
  "tid": "tenant-1234"
}
```

Common Use Cases

- Backend services calling other services
- Scheduled jobs / batch processing
- System-to-system integrations
- No end-user involved

Key Characteristics

- No user involved
- Token represents the application/workload
- Access is based on the application's permissions
- Not affected by user sign-in or user changes
- Example: Microservice calling another API, nightly reconciliation job

	At a Glance	User-Delegated Access	Aspect	Workload Identity
	Acts on behalf of	A signed-in user	Identity	An application / workload
	Authentication	User authentication (interactive)		Application authentication (non-interactive)
	Token subject (sub)	User ID		Application / Service Principal ID
	Consent	User (and admin) consent required		Admin consent (application permissions)
	Typical Flow	Authorization Code Flow (with OIDC)		Client Credentials Flow or Workload Identity Federation
Example		Mobile banking app		Transaction processor service

© 2026 by Innovation In Software Corporation



JSON WEB TOKENS · JWT STRUCTURE AND VALIDATION



HOW A JWT IS USED

1. Client logs in with credentials
2. Auth server authenticates and issues a JWT
3. Client sends the JWT in the Authorization header
4. Resource server validates the token and returns a response



VALIDATION CHECKS

- Signature is valid; token is not expired (exp) or used before its valid time (nbf)
- Audience (aud) is valid; issuer (iss) is trusted; subject (sub) is acceptable



KEY POINTS

- JWTs are stateless -- the server does not store session information
- Never store sensitive data in the payload; always use HTTPS

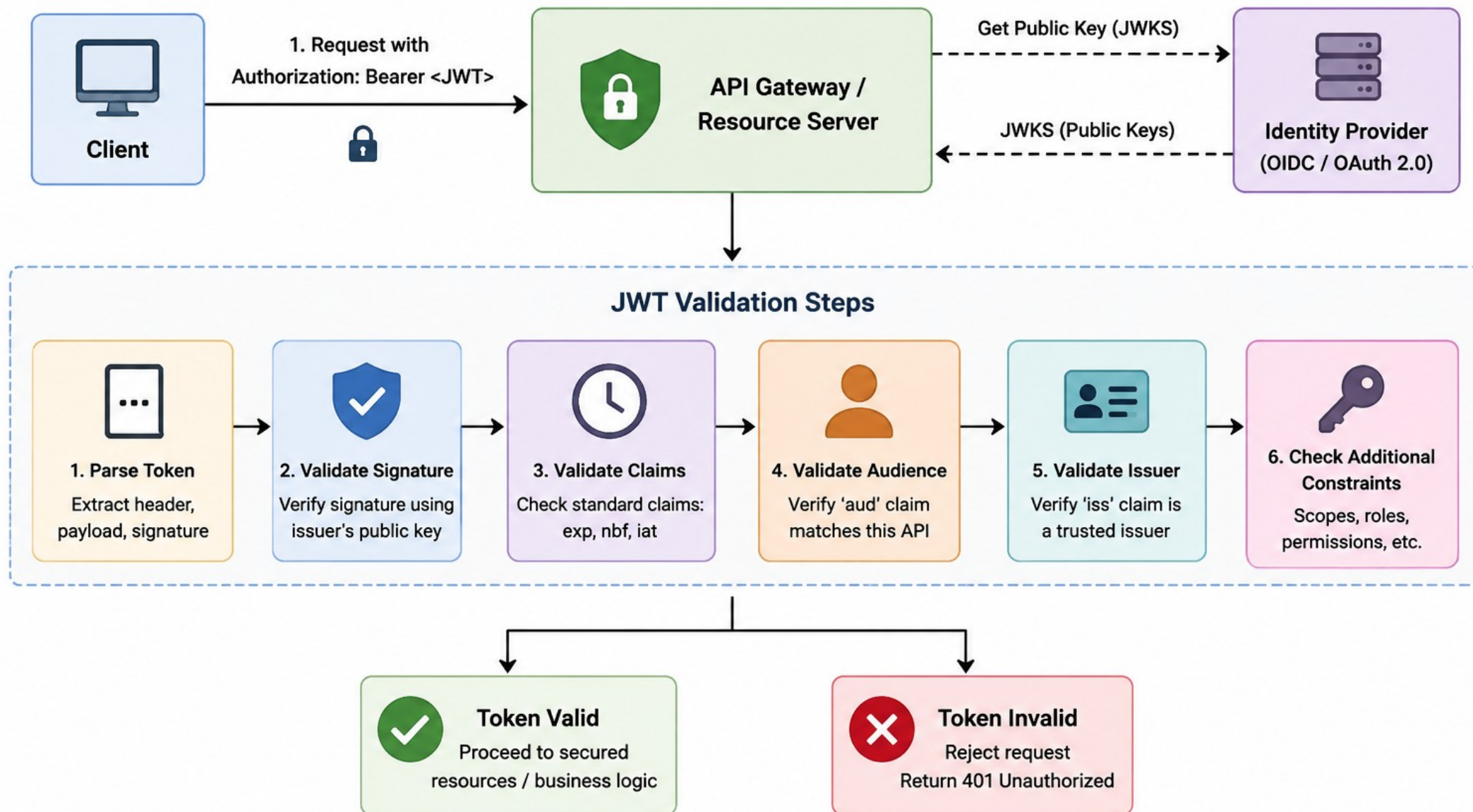


SYMMETRIC VS. ASYMMETRIC SIGNING

- **Symmetric (HMAC):** HS256/384/512, shared secret key, faster
- **Asymmetric (RSA/ECDSA):** RS256/ES256, private key signs, public key verifies -- more secure for distributed systems

KEY TAKEAWAY header.payload.signature -- three Base64Url-encoded parts that let a resource server trust claims without a database lookup. • Split, decode, validate, verify signature, then trust -- five steps stand between an incoming bearer token and an authorized request.

JWT Validation



TOKEN EXPIRATION AND ROTATION · SCOPES, ROLES, AND CLAIMS



COMMON TOKEN LIFETIMES

- **Access token:** 5-15 minutes ID token: 15-60 minutes
Refresh token: days to weeks (rotate on each use)



SCOPES VS. ROLES

- **Scopes:** fine-grained, API-specific, granted to the client application (read:orders, write:orders)
- **Roles:** broader, organization-specific groupings assigned to the user (USER, MANAGER, ADMIN)



ROTATION BENEFITS AND REUSE DETECTION

- Limits damage if a refresh token is stolen; enables detection of token theft
- Reuse of an old refresh token should invalidate the whole token family and force re-authentication



SECURITY NOTE

- Claims are readable by anyone with the token -- never put sensitive data in them
- Use roles to assign permissions; use scopes to limit token capabilities

KEY TAKEAWAY If a rotated refresh token is ever reused, that's a signal of theft -- the entire token family should be revoked immediately. • read:orders is a scope granted to the client; MANAGER is a role assigned to the user -- conflating the two weakens authorization design.

Roles and Scopes

ROLES

Roles define what a user can do
(job functions / responsibilities)



ADMIN

Full system access and
user management



MANAGER

Manage team and
view reports



ANALYST

View data and
generate reports



USER

Basic access and
own data

SCOPES

Scopes define what resources
a user can access (permissions)



read:data

Read access to
data



write:data

Write or update
data



delete:data

Delete data



manage:users

Create, update,
delete users



admin:system

Full system
administration

Roles are assigned Scopes



ADMIN



read:data

write:data

delete:data

manage:users

admin:system



MANAGER



read:data

write:data

manage:users



ANALYST



read:data

write:data



USER



read:data



Effective Access

KEYCLOAK ARCHITECTURE · AZURE AD AND MICROSOFT ENTRA ID INTEGRATION · SECURING SPRING BOOT WITH SPRING SECURITY



KEYCLOAK DATA MODEL

- **Realm:** top-level container for tenants or domains (e.g., 'banking')
- **Client:** an application or service that uses Keycloak (customer-service, account-service)
- **Users, Roles, Groups:** identity, permissions, and logical groupings



KEY COMPONENTS

- Authentication services, authorization services, user federation, token services
- Supports OAuth2, OpenID Connect, and SAML 2.0



WHY IT MATTERS

- Single sign-on, enterprise user identities, group-based access, conditional access
- App registrations define scopes, app roles, redirect URIs, and secrets/certificates



SPRING BOOT CONFIGURATION

- **spring.security.oauth2.resourceserver.jwt.issuer-uri:** `https://login.microsoftonline.com/{tenant-id}/v2.0`
- Validate issuer, audience, and roles/groups claims from the ID and access tokens



HIGH-LEVEL SECURITY FLOW

- User sends request -> filter chain intercepts -> authenticate -> authorize -> access granted/denied

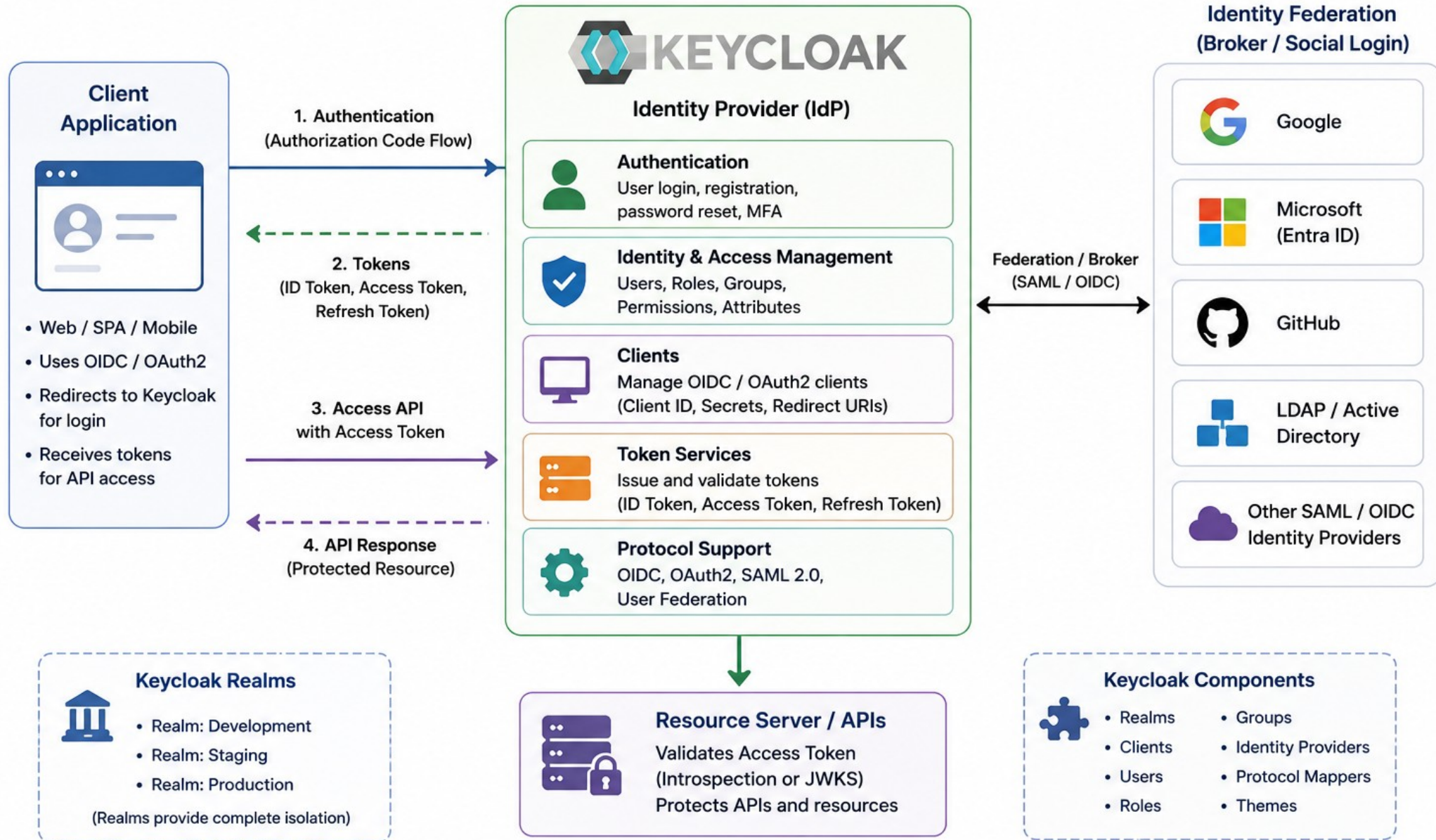


AUTHORIZATION STRATEGIES

- Roles (hasRole), authorities (hasAuthority), method security (@PreAuthorize), URL security
- Disable CSRF for stateless APIs; use stateless session creation policy

KEY TAKEAWAY Realms, clients, users, roles, and groups form Keycloak's data model -- the same building blocks behind every OAuth2/OIDC deployment. • Lecture only — labs use HMAC JWT via `issue-jwt.py`; Entra ID / Keycloak are not a participant install. `issuer-uri` and `jwk-set-uri` configuration in Spring Boot are all it takes to validate tokens issued by an enterprise tenant. • `requestMatchers` rules plus `oauth2ResourceServer().jwt()` turn a Spring Boot service into a token-validating, role-aware protected API in a few lines.

Keycloak / Client IdP Overview



CONFIGURING AN OAUTH2 RESOURCE SERVER

issuer-uri enables OIDC discovery, automatically fetching the JWKS endpoint used to verify every incoming token's signature.



SETUP STEPS

1. Add the spring-boot-starter-oauth2-resource-server dependency
2. Configure spring.security.oauth2.resourceserver.jwt.issuer-uri
3. Configure the security filter chain with authorizeHttpRequests and oauth2ResourceServer(oauth2 -> oauth2.jwt())
4. **Call the API with Authorization:** Bearer <access_token>

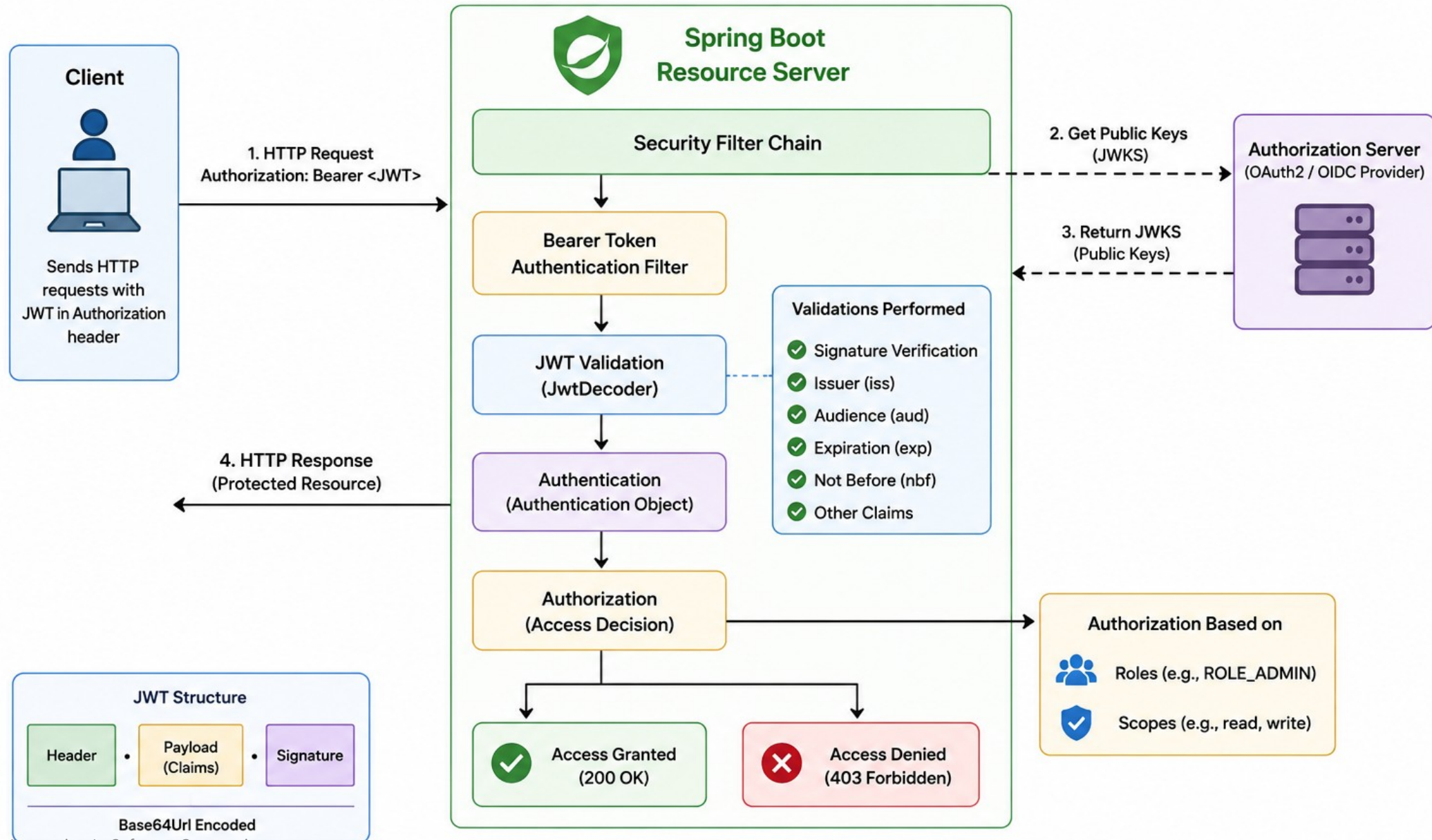


RESPONSE BEHAVIOR

- Invalid or unauthorized tokens return 401 or 403 automatically

KEY TAKEAWAY issuer-uri enables OIDC discovery, automatically fetching the JWKS endpoint used to verify every incoming token's signature.

Spring Security Resource Server



METHOD-LEVEL AUTHORIZATION · ROLE-BASED ACCESS CONTROL · ATTRIBUTE-BASED ACCESS CONTROL



COMMON ANNOTATIONS

- **@PreAuthorize:** evaluates before method execution, blocks access if the expression is false
- **@PostAuthorize:** evaluates after execution, useful for validating the result
- **@PreFilter / @PostFilter:** filter collection arguments or results



BEST PRACTICES

- Use method security for business rules that can't be enforced by URLs alone
- Always combine with authentication and URL-level security for true defense in depth



HOW RBAC WORKS

- Define roles that represent job functions (CUSTOMER, SUPPORT_AGENT, ACCOUNT_MANAGER, ADMIN)
- Assign permissions to each role, then assign roles to users
- Authorization checks a user's role rather than individual permissions



ABAC EVALUATES MULTIPLE ATTRIBUTES

- **Subject attributes:** role, department, clearance level
- **Resource attributes:** sensitivity, owner, classification
- **Environment attributes:** time, location, device, risk score



WHEN TO USE ABAC OVER RBAC

- When access decisions depend on context, not just a static role
- More flexible and fine-grained, at the cost of more complex policy design

KEY TAKEAWAY `hasAuthority('order:read')` or `#id == authentication.name` lets one method enforce both a permission check and object ownership together.

- A `SUPPORT_AGENT` role bundles exactly the permissions that job needs; adding a new agent means assigning one role, not a dozen permissions.
- A policy like 'branch managers can approve loans under \$50k for their own branch' needs ABAC -- RBAC alone can't express that condition.

SERVICE-TO-SERVICE AUTHENTICATION · API GATEWAY SECURITY · CROSS-ORIGIN RESOURCE SHARING



SERVICE-TO-SERVICE AUTH OPTIONS

- OAuth2 client credentials flow -- each service authenticates with its own client ID/secret
- Mutual TLS (mTLS) -- both sides present certificates to verify identity
- Service accounts and platform-issued identities (e.g., OpenShift service accounts)



WHAT CORS CONTROLS

- Which browser origins may call the API, which HTTP methods, and which headers
- The browser enforces this, not the server -- but the server declares the policy



GATEWAY SECURITY RESPONSIBILITIES

- Token validation and authentication before requests reach any backend service
- Rate limiting, IP allowlisting, and WAF-style request filtering
- TLS termination and consistent security headers across every API



BEST PRACTICE

- Restrict allowed origins explicitly in production; never wildcard when credentials are involved

KEY TAKEAWAY OAuth2 client credentials, mTLS, and service accounts each give one service a verifiable identity when calling another. • Centralizing security at the gateway means individual services don't each have to reimplement the same authentication and throttling logic. • Allowing every origin (*) in production is rarely safe -- CORS should be scoped to the specific frontend domains that legitimately call the API.

CSRF AND STATELESS APIS · TRANSPORT LAYER SECURITY · MUTUAL TLS BETWEEN SERVICES



WHY CSRF DOESN'T APPLY TO STATELESS APIS

- CSRF exploits automatic cookie submission by the browser on cross-site requests
- **A stateless API using Authorization:** Bearer <token> is not vulnerable the same way
- `http.csrf(csrf -> csrf.disable())` is standard for JWT-secured, stateless REST APIs



WHY TLS IS NON-NEGOTIABLE

- Encrypts traffic between client and server, preventing eavesdropping and tampering
- Certificate validation confirms the server is who it claims to be
- Enforce TLS 1.2+ everywhere, including internal service-to-service calls



HOW MTLS WORKS

- Client presents a certificate; server presents a certificate; both are validated
- Commonly automated by a service mesh so individual services don't manage certificates directly

KEY TAKEAWAY A bearer token in an Authorization header isn't automatically sent by the browser like a cookie is, which is why stateless APIs don't need CSRF tokens. • Every request in a banking platform -- customer-facing or service-to-service -- must travel over TLS 1.2 or higher, no exceptions. • In a zero-trust internal network, mTLS between services means a compromised network segment still can't impersonate a trusted service.

SECRETS MANAGEMENT FUNDAMENTALS

Database passwords, API keys, and signing keys should be injected at runtime, not baked into the application at build time.



WHAT COUNTS AS A SECRET

- Database passwords, OAuth client secrets, API keys, JWT signing keys, TLS private keys

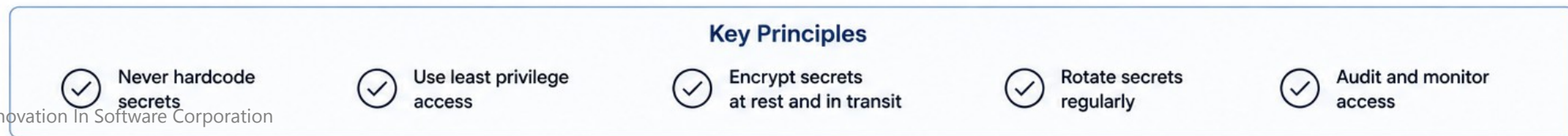
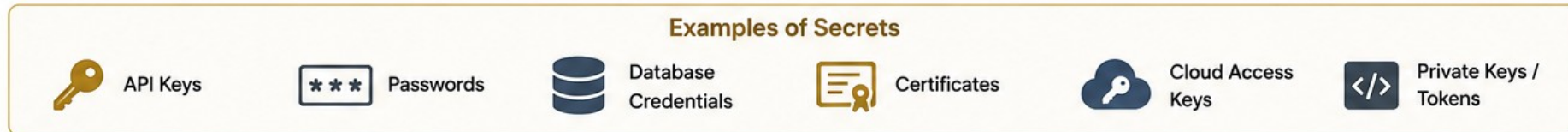


BEST PRACTICES

- Store secrets outside source code, inject at runtime, limit access, rotate regularly
- Use separate secrets per environment and audit secret access

KEY TAKEAWAY Database passwords, API keys, and signing keys should be injected at runtime, not baked into the application at build time.

Secrets Management Basics



OPENSIFT SECRETS · EXTERNAL SECRET STORES AND VAULTS · SECRET ROTATION



OPENSIFT SECRET EXAMPLE

- **apiVersion:** v1, kind: Secret, type: Opaque
- **stringData:**
ACCOUNT_DB_PASSWORD: <injected securely>
- Real secret creation should follow enterprise security procedures, not ad hoc values



COMMON EXTERNAL SECRET STORES

- HashiCorp Vault, Azure Key Vault, AWS Secrets Manager, Google Secret Manager
- Provide dynamic secrets, fine-grained access policies, and centralized audit logging



WHEN TO ROTATE SECRETS

- Periodically, after suspected exposure, when team members change roles
- When service accounts are replaced, or after a security incident

KEY TAKEAWAY A Secret of type Opaque, mounted as an environment variable, keeps ACCOUNT_DB_PASSWORD out of application.yml entirely.

- An external vault can issue short-lived, on-demand database credentials instead of one static password shared across every service instance.
- Rotating periodically, after a suspected exposure, or when a team member changes roles limits the blast radius of any single leaked credential.

API INPUT VALIDATION · OWASP API SECURITY RISKS · RATE LIMITING AND ABUSE PREVENTION



WHAT TO VALIDATE

- Path variables, query parameters, request bodies, headers where relevant, file uploads



TOP OWASP API RISKS

- Broken object level authorization, broken authentication, broken object property level authorization
- Unrestricted resource consumption, broken function level authorization
- Security misconfiguration, improper inventory management



BEST PRACTICES

- Validate all inputs on the server side; use allowlists, not denylists
- Use parameterized queries; validate files by type, size, and content



WHERE AND WHY

- **Apply limits close to the entry point:** API gateway, load balancer, or service mesh
- Use different limits for different clients or operations (login vs. balance lookup vs. payment)
- Return clear 429 responses and monitor rate-limit rejections

KEY TAKEAWAY Unvalidated input is the root cause behind injection attacks, broken object authorization, and countless other API vulnerabilities.

- Customer 101 changing a URL to /accounts/5002 and getting another customer's data is exactly the failure OWASP's list is built around.
- 429 Too Many Requests is the standard signal that tells a well-behaved client to back off -- applied at the gateway, closest to the entry point.

Data Classification, Minimization, and Log Redaction

1. Data Classification

Classify data based on sensitivity and impact

 Critical	Highest sensitivity Highest impact	SSN, Payment Details, Health Records
 Confidential	High sensitivity High impact	PII, Credentials, Personal Data
 Internal	Moderate sensitivity Moderate impact	Employee Info, Business Data
 Public	Low sensitivity Low impact	Marketing Info, Public Content

Classification Guidelines



Identify the type of data



Assess sensitivity and business impact



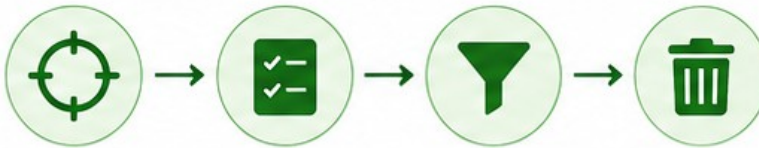
Assign appropriate classification level



Review and update classifications regularly

2. Data Minimization

Collect, process, and retain only what is necessary



Define Purpose

Clearly define the purpose for collecting data

Assess Necessity

Collect only the data essential for the purpose

Minimize Data

Limit data to the minimum required

Remove / Retain Appropriately

Delete or anonymize data no longer needed

Minimization Principles

- ✓ Collect only necessary data
- ✓ Avoid collecting data "just in case"
- ✓ Use aggregated or anonymized data when possible
- ✓ Set clear retention periods
- ✓ Regularly review and purge unnecessary data

Benefits



Reduces Risk



Lowers Storage Costs



Simplifies Compliance



Builds Trust

3. Log Redaction

Prevent sensitive data from being written to logs

Redact Before Log



Original Log
(With Sensitive Data)

userId=12345
email=john.doe@example.com
ssn=123-45-6789
token=eyJhbGciOiJIUzI1NiIs...



Redaction Engine



Redacted Log
(Safe to Store)

userId=12345
email=***@example.com
ssn=***-**-6789
token=***REDACTED***

What to Redact



PII
(Personally Identifiable Information)



Credentials
& Secrets



Payment
Information



Tokens &
API Keys



Health
Information

Redaction Best Practices

- ✓ Maintain a list of sensitive data patterns and fields
- ✓ Redact at the application level, not just in log aggregation
- ✓ Use consistent masking (e.g., *** or partial masking)
- ✓ Never log secrets, tokens, or full raw payloads
- ✓ Regularly test logs to ensure no sensitive data leaks

SECURITY LOGGING AND AUDIT TRAILS · DATA ENCRYPTION AT REST AND IN TRANSIT · PERSONALLY IDENTIFIABLE INFORMATION



WHAT TO LOG

- Failed login attempts, access denied events, account updates
- Payment creation, admin operations, secret rotation, high-risk AI tool calls



TWO ENCRYPTION DOMAINS

- **In transit:** TLS 1.2+ for all client and service-to-service traffic
- **At rest:** encrypted databases, encrypted disks, and encrypted backups
- Use secure key management -- never hardcode encryption keys



WHAT NEVER TO LOG

- Passwords, tokens, full account numbers, secret values, sensitive personal data



PROTECTING PII

- Minimize collection of sensitive data to only what's necessary
- Mask, tokenize, or pseudonymize PII when full values aren't required
- Never log PII, and encrypt it both at rest and in transit

KEY TAKEAWAY Audit logs answer 'who did what, when' after an incident -- but only if they were captured consistently before the incident happened. • TLS protects data in transit; database and disk-level encryption protects data at rest -- a complete strategy needs both, not just one. • Names, addresses, SSNs, and account numbers are exactly the data a banking platform must protect most carefully across every system that touches it.

SECURITY AND COMPLIANCE CONTROLS · THREAT MODELING MICROSERVICES · ZERO-TRUST ARCHITECTURE



WHY COMPLIANCE CONTROLS MATTER

- Regulatory and contractual obligations (e.g., GDPR, HIPAA-style requirements) shape required controls
- Controls must be documented, testable, and auditable -- not just implemented



ZERO-TRUST PRINCIPLES

- Verify explicitly, use least-privilege access, assume breach
- Reduce attack surface, continuously monitor and re-validate



THE THREAT MODELING PROCESS

1. Define scope
2. Create architecture view
3. Identify threats (STRIDE)
4. Assess risks
5. Mitigate & validate
6. Monitor & iterate



BEST PRACTICES

- Model early and often; include automated checks in CI/CD pipelines
- Prioritize risks based on business impact; validate controls with testing (SAST/DAST, pen tests)



ZERO TRUST IN MICROSERVICES

- mTLS between services, fine-grained RBAC/ABAC authorization
- API gateway with policy enforcement, observability across traces, logs, and metrics

KEY TAKEAWAY Compliance frameworks translate 'be secure' into specific, auditable controls a banking platform must demonstrably satisfy. • STRIDE -- spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege -- gives every threat a name and a category. • Never trust, always verify, least privilege, assume breach -- four principles that replace perimeter-based security with continuous verification.



Secure Coding Checklist

Build secure by design. Protect data. Prevent vulnerabilities. Earn trust.

Follow secure coding principles and validate your code at every step to reduce risk and protect your microservices and users.



Protect Data
Keep data safe and private



Reduce Risk
Prevent vulnerabilities and breaches



Meet Compliance
Align with OWASP, standards, and laws



Improve Quality
Write reliable, secure, maintainable code



Build Trust
Earn customer and user confidence

1. INPUT VALIDATION



Validate all inputs on the server side.

- ✓ Validate all user inputs (type, length, format, range)
- ✓ Use allowlists, not denylists
- ✓ Sanitize or encode outputs in the correct context
- ✓ Use parameterized queries / prepared statements
- ✓ Validate files (type, size, content) before upload

Why: Prevents injection attacks, XSS, and other input-related vulnerabilities.

2. AUTHENTICATION & AUTHORIZATION



Verify identity and enforce appropriate access.

- ✓ Use strong authentication (MFA, SSO when possible)
- ✓ Store passwords using strong hashing (e.g. BCrypt)
- ✓ Never hardcode credentials or secrets
- ✓ Enforce least privilege and role-based access
- ✓ Validate tokens and sessions on the server
- ✓ Re-authenticate for sensitive actions

Why: Prevents unauthorized access and privilege escalation.

3. DATA PROTECTION



Protect data at rest, in transit, and in use.

- ✓ Encrypt data in transit (TLS 1.2+)
- ✓ Encrypt sensitive data at rest
- ✓ Never log sensitive data (PII, secrets, tokens)
- ✓ Use secure key management
- ✓ Minimize collection of sensitive data
- ✓ Mask, tokenize, or pseudonymize when possible

Why: Protects confidentiality and reduces impact of data breaches.

4. ERROR HANDLING & LOGGING



Handle errors securely and log responsibly.

- ✓ Do not expose stack traces or internal details in responses
- ✓ Return generic error messages to users
- ✓ Log security-relevant events
- ✓ Use structured logging with correlation IDs
- ✓ Avoid logging sensitive data

Why: Prevents information leakage and supports incident response.

5. DEPENDENCIES & COMPONENTS



Use secure and up-to-date components.

- ✓ Keep dependencies and libraries up to date
- ✓ Scan for known vulnerabilities (SCA tools)
- ✓ Remove unused dependencies
- ✓ Use trusted and official sources
- ✓ Review third-party licenses and risks

Why: Outdated or vulnerable components are a common attack vector.

6. SESSION MANAGEMENT



Manage sessions securely.

- ✓ Use secure, HTTP-only, SameSite cookies
- ✓ Set appropriate session timeout
- ✓ Regenerate session ID after login
- ✓ Invalidate sessions on logout
- ✓ Avoid URL rewriting for session IDs

Why: Prevents session hijacking and fixation attacks.

7. COMMUNICATION SECURITY



Secure service-to-service and client communication.

- ✓ Use TLS/mTLS for all internal and external calls
- ✓ Validate certificates and hostnames
- ✓ Set timeouts and retries to avoid DoS
- ✓ Avoid sending sensitive data in URLs
- ✓ Use secure messaging (sign/encrypt) when needed

Why: Protects data integrity and confidentiality across the network.

8. CODE QUALITY & BEST PRACTICES



Write clean, simple, and maintainable code.

- ✓ Follow coding standards and secure patterns
- ✓ Apply least privilege and separation of concerns
- ✓ Avoid hardcoded secrets and magic values
- ✓ Use static analysis (SAST) in CI/CD
- ✓ Review code (peer review) regularly

Why: Reduces defects, simplifies reviews, and prevents security issues early.

ADDITIONAL BEST PRACTICES



Follow OWASP Top 10



Threat model your services



Automate security tests in CI/CD



Monitor, detect, and respond



Train developers continuously

REMEMBER



Security is a shared responsibility.
Build security in, not on.
Small habits. Big impact.

USEFUL REFERENCES

- OWASP Top 10 – <https://owasp.org/www-project-top-ten/>
- OWASP Cheat Sheets – <https://cheatsheetseries.owasp.org/>
- CWE (Common Weakness Enumeration) – <https://cwe.mitre.org/>
- NIST Secure Software Development Framework (SSDF)



Incident Response Considerations

Prepare. Detect. Respond. Recover. Learn.



No organization can prevent every incident. A strong incident response capability minimizes impact, speeds recovery, and strengthens future defenses.



Reduce Impact
Contain incidents quickly and limit damage



Ensure Business Continuity
Restore critical services as fast as possible



Meet Compliance
Fulfill legal, regulatory, and contractual obligations



Build Trust
Communicate transparently with stakeholders



Improve Over Time
Learn from incidents and strengthen security posture

INCIDENT RESPONSE LIFECYCLE



1. Preparation
Build capability before incidents happen.



2. Detection & Analysis
Detect potential incidents and analyze to confirm and assess impact.



3. Containment
Limit the scope and prevent further damage.



4. Eradication & Recovery
Remove the threat, restore systems, and validate.



5. Lessons Learned
Document findings and improve processes, controls, and training.

KEY CONSIDERATIONS



People & Roles
Define an incident response team with clear roles and escalation paths.



Communication Plan
Establish internal and external communication channels and templates.



Business Impact
Understand critical services, dependencies, and recovery priorities.



Evidence Preservation
Preserve logs, system images, and artifacts for analysis and compliance.



Legal & Regulatory
Know reporting obligations (e.g., GDPR, HIPAA) and legal requirements.



Third-Party Coordination
Coordinate with vendors, cloud providers, and law enforcement when needed.

COMMON INCIDENT TYPES



Malware / Ransomware
Malicious software that disrupts or encrypts systems and data.



Unauthorized Access
Compromise of accounts, credentials, or privileged access.



Data Breach / Exfiltration
Unauthorized access or theft of sensitive data.



Denial of Service (DoS)
Attacks that make services unavailable.



Insider Threat
Malicious or negligent actions by internal users.



Vulnerabilities Exploited
Exploitation of software, configuration, or known weaknesses.

INCIDENT RESPONSE PLAN ESSENTIALS

Preparation



- Risk assessment and scenario planning
- Tools, runbooks, and access readiness
- Training and awareness programs

Detection & Analysis



- Continuous monitoring and alerting
- Triage and validate incidents
- Determine scope and impact

Containment



- Short-term: isolate affected systems
- Long-term: implement fixes and blocks
- Communicate status to stakeholders

Eradication & Recovery



- Remove malware or attacker access
- Patch vulnerabilities and restore from backups
- Validate integrity and monitor closely

Lessons Learned



- Root cause analysis
- Document findings and actions
- Update controls, policies, and training

communications during an incident

INTERNAL

- Executive / Leadership
- IT / Security Teams
- Operations / Dev Teams
- Legal & Compliance
- HR (if people impacted)



EXTERNAL

- Customers / Users
- Regulators (as required)
- Law Enforcement
- Vendors / Partners
- Public / Media (if needed)

POST-INCIDENT REVIEW CHECKLIST

- ✓ What happened?
- ✓ Why did it happen?
- ✓ What was the impact?
- ✓ What worked well?
- ✓ What didn't work?
- ✓ What controls need improvement?
- ✓ What action items will we track?
- ✓ How will we prevent recurrence?

DETECTION & MONITORING ENABLERS



Centralized Logging



SIEM & Alerting



Threat Intelligence Feeds



Vulnerability Scanning



Endpoint Detection & Response



User Behavior Analytics

BEST PRACTICES

- ✓ Test the plan regularly
- ✓ Run tabletop exercises
- ✓ Automate where possible
- ✓ Keep systems and backups updated
- ✓ Apply least privilege access
- ✓ Segment networks
- ✓ Monitor continuously
- ✓ Improve and adapt



REMEMBER



Speed, coordination, and communication are critical. A well-prepared team makes the difference between a minor event and a major breach.



Have a Plan.
Stay Prepared.
Respond Together.



Every Minute Counts.
Detect early. Act fast. Limit impact.



Learn and Improve.
Stronger today. More resilient tomorrow.

EXERCISE 3.2 — SECURE AN ENDPOINT WITH JWT ROLES/SCOPES

Module 7 checkpoint B — Roles/scopes on protected APIs.



OBJECTIVE

- Roles/scopes on protected APIs
- 10 min · code warmup checkpoint
- **Worksheet:**
curriculum/day-03/exercises/exercise-3.2-jwt-roles-scopes.md



DO THIS

1. No Bearer token → 401. Wrong scope → 403 (not a silent 200)
2. **teller:** accounts.read/write. ops: transactions + accounts.read. readonly: *.read
3. Transaction Service forwards the caller's JWT when it GETs Account Service
4. Never log Authorization headers or JWT claims



EXPECTED RESULT

- A scope matrix you can test with issue-jwt.py teller / ops / readonly.
- Lab 3 uses issue-jwt.py (HMAC). Keycloak / Azure AD are lecture only.

KEY TAKEAWAY A scope matrix you can test with issue-jwt.py teller / ops / readonly.